STRATIFIED SOURCE REALISM · MODEL 4.0

CHRONOLOGICAL REALIGNMENT ARRAY

A conceptual engineering manual in mathematical physics, order theory, and formal systems design

System designation: Least-Fixed-Point Chronological Realignment Array

Document class: Theoretical / speculative system design

Edition: Model v4.0 · 15 October 2026

HONEST BOUNDARY
The lattice and fixed-point mathematics in this manual are established mathematical concepts when their assumptions are met. Their interpretation as enabling timeline displacement is a speculative metaphysical extension. No validated physical mechanism for a working time machine is established here.

Prepared as a structured, auditable reference for conceptual modelling, computational prototyping, and rigorous separation of theorem, hypothesis, and physical evidence.

DOCUMENT CONTROL & READING GUIDE

Document control	Specification
Model designation	SSR–CRA v4.0
System name	Least-Fixed-Point Chronological Realignment Array
Primary disciplines	Order theory, mathematical logic, spacetime physics, information theory
Architecture status	Conceptual model; not a validated physical machine
Mathematical foundation	Complete lattices, monotone operators, transfinite iteration
Engineering status	No demonstrated mechanism for timeline displacement
Governing principle	Honest Boundary: formal validity is distinct from physical realisation

Governing engineering notice — Honest Boundary
The mathematical results concerning complete lattices and monotone operators can be stated rigorously. The interpretation of those results as enabling alternate timelines, higher-dimensional access, or chronological displacement is a speculative metaphysical extension of Stratified Source Realism (SSR). No experimentally validated mechanism currently establishes that lattice manipulation can alter a physical history, transfer an observer between timelines, or produce a working time machine.

This manual defines a mathematically explicit conceptual architecture, with testable software-level components and clearly identified hypothetical interfaces. Its diagrams and component specifications are not fabrication-ready hardware blueprints.

Notation conventions

• Established mathematics: definitions and theorems stated under their explicit assumptions.

• Conceptual SSR layer: proposed interpretation of formal structures as strata or actuality representations.

• Physical claim: requires a defined mechanism, measurable predictions, and independent evidence.

• Engineering tolerance: meaningful only when the quantity, units, measurement method, and acceptance criteria are declared.

SECTION 1 — SPECIFICATION & METAPHYSICAL BASELINE

1.1 System definition

The Least-Fixed-Point Chronological Realignment Array (CRA) is a proposed SSR architecture intended to formalise relationships between admissible structures, consistency constraints, actuality representations, and observer-relative temporal descriptions.

It is not defined as a propulsion system. Its conceptual operation is expressed as a sequence of transformations over a space of formal states.

Level	Description
1. Formal state evolution	Mathematically defined operations on a lattice.
2. Representational realignment	Transformations between abstract descriptions of states.
3. Physical chronological displacement	A hypothesised phenomenon for which no established implementation is known.

The first can be investigated mathematically and computationally. The second can be studied as a formal modelling problem. The third remains an unverified physical hypothesis.

1.2 SSR stratification model

The labels M0 through M7+ identify proposed descriptive strata, not experimentally established physical dimensions.

Stratum	Conceptual designation	Formal role in CRA
M0	Foundational primitives	Basic symbols, values, and admissibility rules
M1	Relational structure	Relations, orderings, and elementary dependencies
M2	State configurations	Sets of admissible configurations
M3	Physical spacetime representation	Local spacetime states and observer-indexed histories
M4	Quantum/informational representation	Abstract information states and proposed cross-level mappings
M5	Unified structural representation	Higher-level relations between admissible models
M6	Global consistency layer	Compatibility conditions across representations
M7+	Extended meta-structure	Further abstract levels, if independently defined

Calling M4 or M5 a higher informational space does not establish that either is a physically accessible dimension. Their relationships to spacetime are represented by mathematical maps whose existence and properties must be specified rather than assumed.

1.3 Governing SSR expression

$$\mathbf{O = \mu\, \Gamma_C}$$

$$\mathbf{O = \mu\, X\, .\, \Gamma_{\leq,R,C}(X)}$$

where:

- O denotes the proposed totality of admissible structures.
- Γ_C denotes a generative operator subject to consistency constraints C.

• R denotes additional admissibility or relational constraints.

• μ denotes the least fixed point of the specified operator, when the relevant mathematical conditions are satisfied.

$$\Gamma_C(O) = O$$

This says that applying the operator to the resulting structure produces no further change. It does not, by itself, say that O is the physical universe, that multiple timelines exist, or that an observer can move between them.

1.4 Lattice architecture

Let U be a specified universe of formal elements. Define L = P(U), where P(U) is the power set of U, ordered by subset inclusion:

$$L = P(U),\ X \leq Y \text{ iff } X \subseteq Y$$

Then (L, ⊆) is a complete lattice. For every family A contained in L:

$$\bigvee A = \bigcup_{X \in A} X \text{ and } \bigwedge A = \bigcap_{X \in A} X$$

$$\bot = \varnothing \text{ and } \top = U$$

For a monotone operator $\Gamma_C : L \to L$:

$$X \subseteq Y \Rightarrow \Gamma_C(X) \subseteq \Gamma_C(Y)$$

the Knaster–Tarski theorem guarantees that the fixed points of Γ_C form a complete lattice and that a least fixed point exists:

$$\mathrm{lfp}(\Gamma_C) = \bigcap \{ X \in L : \Gamma_C(X) \subseteq X \}$$

The intersection is taken over all pre-fixed points of the operator. The theorem establishes a mathematical existence result. It does not supply a physical actuator, a spacetime metric, an energy source, or a causal mechanism.

1.5 Proposed chronological-realisation hypothesis

$$M5 \xrightarrow{r_{5,4}} M4 \xrightarrow{r_{4,3}} M3$$

Here $r_{m,n}$ denotes a proposed restriction or representation map from stratum M_m to M_n.

Formal state transition ≠ Change of physical state ≠ Travel to another time

The model defines an interface for investigating chronological realignment without claiming that the interface is already physically realisable.

SECTION 2 — CRISP ASCII ENGINEERING SCHEMATICS

2.1 Diagram A: System Block Component Schematic

The schematic represents a conceptual system boundary, with physical monitoring and isolation equipment separated from hypothetical mathematical operators.

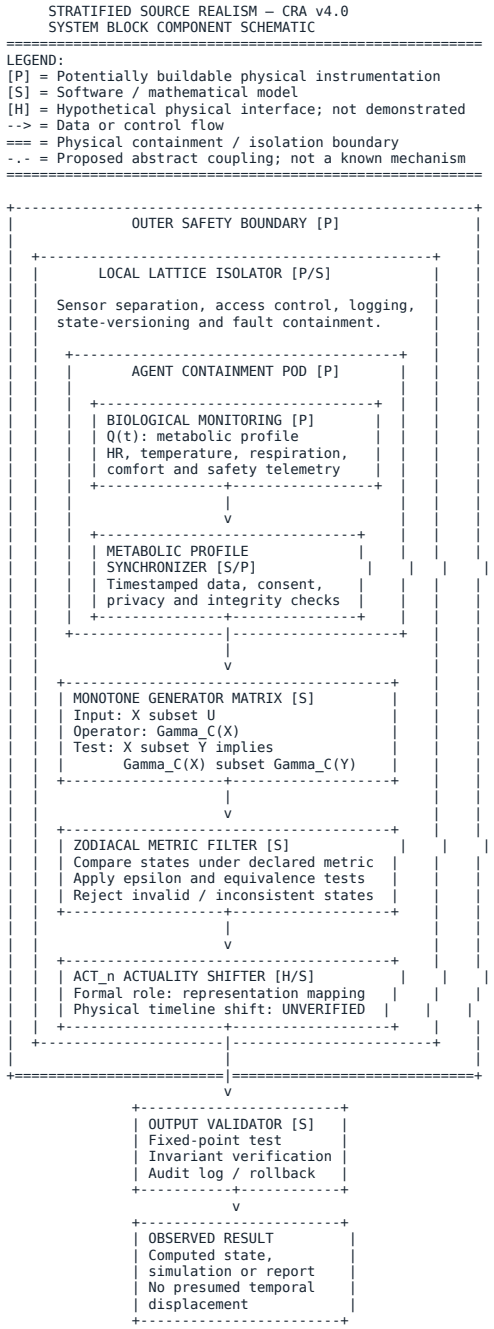
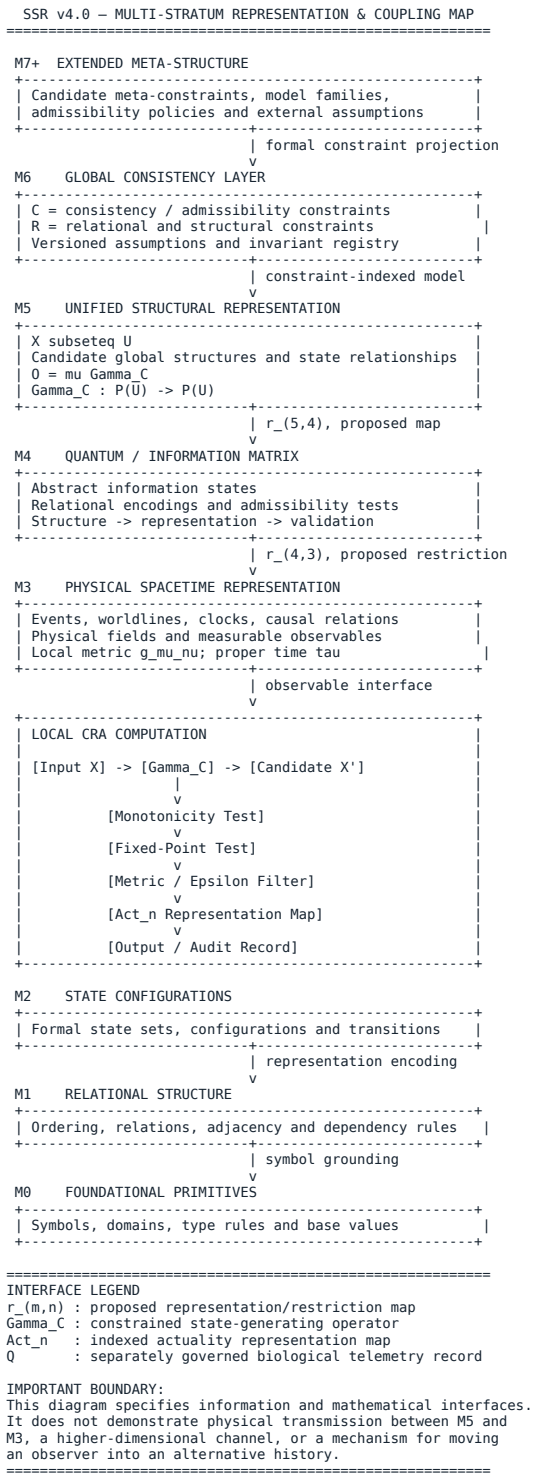


Diagram A — Engineering interpretation

Component	Design role	Status
Agent Containment Pod	Physical enclosure and environmental monitoring for a human participant	Conventional engineering concept
Metabolic Profile Synchronizer	Collects and time-aligns consented physiological measurements	Conventional data-processing concept
Local Lattice Isolator	Isolates software state, experimental inputs, and control interfaces	Analogy to fault containment; not a physical lattice shield
Monotone Generator Matrix	Executes a monotone operator over a represented state space	Software implementation is possible
Zodiacal Metric Filter	Evaluates a specified metric and declared equivalence criteria	Requires a defined state space and metric
Act_n Actuality Shifter	Maps a representation at one index to another	Formal mapping possible; physical chronology shift unverified
Output Validator	Checks invariants, fixed-point conditions, and logs	Conventional software engineering

Safety rule: biological monitoring must not be interpreted as measuring, protecting, or transporting subjective experience between timelines. Q is a data model for physiological observations; it is not a complete representation of consciousness.

2.2 Diagram B: Comprehensive Signal Fluidity & Coupling Map



2.3 Cross-level coherence condition

$$\mathbf{Act_n(r_\{m,n\}(x)) = q_\{m,n\}(Act_m(x))}$$

- x belongs to a declared domain D_m.
- r_{m,n}: D_m → D_n is a restriction or representation map.
- Act_m and Act_n are indexed maps.
- q_{m,n} maps the corresponding actuality representations.

The equality requires compatible domains and codomains. It can be tested for finite models or formally proved under specified assumptions. It does not establish that the mapped representations correspond to independently

existing physical timelines.

SECTION 3 — TRANSFINITE ITERATIVE ALGORITHMS

3.1 Mathematical prerequisites

Let L = P(U), ordered by inclusion, and let Γ_C: L → L be a monotone operator.

For all X,Y in L: X ⊆ Y ⇒ Γ_C(X) ⊆ Γ_C(Y)

A monotone operator on a complete lattice has a least fixed point by the Knaster–Tarski theorem. For the transfinite construction, begin with the bottom element.

3.2 Core convergence sequence

$X_0 = \emptyset$
$X_{\alpha+1} = \Gamma_C(X_\alpha)$
$X_\lambda = \bigcup_{\beta < \lambda} X_\beta$ for limit ordinals λ

The union at a limit stage is the supremum of the earlier stages in the powerset lattice.

Successor and limit stages

At a successor ordinal, apply the operator once to the preceding state. At a limit ordinal, take the union of all preceding states. These equations define the iteration; they do not mean that a conventional computer can literally execute an unbounded sequence of ordinal-indexed operations.

Monotonicity and inflationarity are distinct

Monotonicity: X ⊆ Y ⇒ Γ_C(X) ⊆ Γ_C(Y)
Inflationarity: X ⊆ Γ_C(X)

Monotonicity means preservation of order. Inflationarity means every state is contained in its image. They are different properties. With bottom-start iteration, monotonicity makes the generated sequence ascending, but does not make Γ_C inflationary on every element of L.

3.3 Exact convergence criterion

Converged(α) iff Γ_C(X_α) = X_α
Finite model: Converged_finite(k) iff X_{k+1} = X_k

When equality holds, the result is a fixed point. Under the stated monotonicity and bottom-start assumptions, it is the least fixed point. In a finite representation, equality can be checked exactly; symbolic implementations require documented semantics.

For a monotone operator on a set-sized complete lattice, the ascending transfinite iteration from bottom eventually stabilises. A strictly ascending chain cannot continue assigning distinct elements of L to ordinals beyond the cardinality of L. Once stable, the successor equation gives the fixed-point condition. Existence does not imply computational efficiency.

3.4 Reference algorithm for a finite model

```
def least_fixed_point(bottom, gamma, max_iterations):
    """
    Compute a fixed point by iteration from the bottom element.
    gamma must implement a monotone endofunction.
    """
    current = bottom

    for iteration in range(max_iterations + 1):
        candidate = gamma(current)

        if candidate == current:
            return current, iteration

        current = candidate

    raise RuntimeError(
        "No fixed point was reached within the iteration bound."
    )
```

This implementation assumes that gamma returns valid lattice elements and is well-defined. It does not automatically prove monotonicity. For a finite powerset L = P(U), exhaustive checking is possible when U is sufficiently small:

$$\textbf{For all } X,Y \subseteq U\textbf{: } X \subseteq Y \Rightarrow \Gamma_C(X) \subseteq \Gamma_C(Y)$$

For large or symbolic state spaces, proofs, property-based tests, or verified operator construction may be more practical.

3.5 Epsilon-tolerant Zodiacal Metric Filtering

Let S_i be the state space at index i; $f_i: S_i \to Z_i$ a feature or observable map; $d_i: Z_i \times Z_i \to [0,\infty)$ a metric; and $\varepsilon_i \geq 0$ a declared tolerance.

$$d_i(f_i(s), f_i(s')) \leq \varepsilon_i$$

This means the feature representations are within the chosen tolerance. It does not, by itself, imply physical equivalence, entropy reduction, or preservation of a timeline.

$$s \sim_{\{\varepsilon_i\}} s' \text{ iff } d_i(f_i(s), f_i(s')) \leq \varepsilon_i$$

> A threshold relation generated by a metric is generally reflexive and symmetric but not necessarily transitive. It is therefore not automatically an equivalence relation.

If the consistency policy requires a genuine equivalence relation $\sim_C$, define it independently and use the metric test as an additional acceptance condition:

$$\text{Accept}_i(s,s') \text{ iff } (s \sim_C s') \text{ and } d_i(f_i(s), f_i(s')) \leq \varepsilon_i$$

Filtering algorithm

```
def zodiacal_metric_filter(
    state_a, state_b, feature_map, distance,
    epsilon, equivalent_under_C
):
    """Check declared equivalence and an epsilon-distance bound."""
    if epsilon < 0:
        raise ValueError("epsilon must be non-negative")

    if not equivalent_under_C(state_a, state_b):
        return False

    feature_a = feature_map(state_a)
    feature_b = feature_map(state_b)

    return distance(feature_a, feature_b) <= epsilon
```

What the filter can and cannot establish

Claim	Status
Feature vectors fall within a defined tolerance	Directly testable
A declared equivalence relation holds	Testable or provable, depending on representation
The candidate satisfies a specified consistency policy	Testable if the policy is formalised
Entropic dissipation has been screened out	Requires a defined thermodynamic model and entropy criterion
The candidate represents an alternative physical timeline	Not established by the metric test
The candidate has been physically reached	Requires independent empirical evidence

To screen for thermodynamic constraints, a model needs a defined entropy function S_{th}, an identified physical system, and an appropriate balance law. A small feature-space distance alone cannot demonstrate zero or negative entropy production.

3.6 Closure-operator extension

If the intended operator is specifically a closure operator, define and verify all three properties:

Extensivity: $X \subseteq \Gamma_C(X)$

Monotonicity: $X \subseteq Y \Rightarrow \Gamma_C(X) \subseteq \Gamma_C(Y)$

Idempotence: $\Gamma_C(\Gamma_C(X)) = \Gamma_C(X)$

Together, these define a closure operator on the ordered set. This is stronger than the monotonicity required by Knaster–Tarski. A closure operator already maps each state to a closed state in one application, so repeated iteration becomes unnecessary after that application. If CRA intends a genuinely iterative construction, it must explain why a general monotone operator is appropriate instead.

SECTION 4 — MECHANICAL COMPONENT LOGISTICS

4.1 Bill of Materials: status and interpretation

The following bill of materials is a logical and software architecture register, not a procurement list for a machine capable of chronological displacement. Physical items are included only where they serve an independently established measurement, computing, or safety function.

The labels M0–M7+ designate conceptual interaction layers, not calibrated physical dimensions. A numerical engineering tolerance is meaningful only when its measured quantity, units, test method, operating conditions, and acceptance criteria are specified. Where these are unavailable, the tolerance is a formal verification requirement rather than an invented physical value.

4.2 Subsystem A — Monotone processing and formal verification

Formal component	Layer	Required tolerance / acceptance criterion	Functional purpose
Monotone Inductive Processing Core	M0–M2	Exact order preservation for all tested pairs; proof required for unrestricted domain	Implements Γ_C over a declared state space
Lattice Domain Registry	M0–M2	Exact type and membership consistency	Defines U, L, bottom, and top
Constraint Evaluation Kernel	M1–M6	Every rule versioned; no silent rule substitution	Evaluates consistency policy C
Fixed-Point Convergence Monitor	M2–M6	Exact equality for finite representations; documented semantics for symbolic states	Detects Γ_C(X) = X
State-Version and Audit Ledger	M0–M7+	No unlogged state mutation; integrity checks pass	Preserves reproducibility and transition history

Central acceptance condition:

$$X \subseteq Y \Rightarrow \Gamma_C(X) \subseteq \Gamma_C(Y)$$

For a finite domain, this can be checked exhaustively. For a larger or symbolic domain, a mathematical proof or verified implementation strategy is required. No electronic processor physically manipulates an abstract lattice merely because its software calculates lattice operations.

4.3 Subsystem B — Cross-layer representation and actuality maps

Formal component	Layer	Required tolerance / acceptance criterion	Functional purpose
Cross-Stratum Map Registry	M3–M7+	Explicit domain and codomain for every map	Records permitted representation relationships
Restriction Map r_m,n	M_m to M_n	Type correctness; declared preservation properties	Maps higher-level representations to lower-level ones
Act_n Actuality Translation Module	M0–M7+	Coherence equation verified for the specified domain	Computes an indexed formal representation
Coherence Verification Engine	M3–M7+	Equality proven or exhaustively checked over finite model	Detects incompatible cross-level mappings
Physical Observation Adapter	M3	Instrument-specific uncertainty and calibration limits	Connects model outputs to measured observables

$$Act_n(r_{m,n}(x)) = q_{m,n}(Act_m(x))$$

This is a coherence constraint, not a hardware calibration equation. It specifies what the maps must satisfy if the model is internally consistent. It does not demonstrate that the maps can be implemented as physical

waveguides or that they can change an observer's location in time.

Act_n Actuality Translation Waveguides

The term waveguide is retained as an architectural metaphor. In ordinary engineering, a waveguide is a physical structure that confines or guides electromagnetic or other supported waves. No known waveguide is established to guide an observer between alternative histories through an abstract fixed-point operator.

For an implementable software model, replace the physical-waveguide interpretation with:

- a typed mapping interface;
- a validated input/output schema;
- a coherence checker;
- a versioned mapping registry;
- a rollback mechanism for invalid outputs.

Any future physical implementation claim would require a separate mechanism, governing equations, predicted measurements, and independent experimental validation.

4.4 Subsystem C — Metric filtering and state integrity

Formal component	Layer	Required tolerance / acceptance criterion	Functional purpose
Zodiacal Metric Filter	M2–M6	$d_i(f_i(s), f_i(s')) \leq \varepsilon_i$	Screens candidate states against a declared distance criterion
Metric Definition Registry	M0–M4	Non-negativity, identity of indiscernibles, symmetry, triangle inequality if called a metric	Defines state-comparison geometry
Equivalence Relation Validator	M1–M6	Reflexivity, symmetry, transitivity	Checks $\sim_C$ independently of threshold distance
Entropy Model Adapter	M3–M5	Explicit thermodynamic state model and entropy definition	Tests thermodynamic claims without conflating them with metric similarity
Transition Safety Gate	M2–M6	Rejects missing, invalid, or inconsistent inputs	Prevents unverified candidate states from being accepted

The phrase Zodiacal Metric has no standard, universally accepted definition in mathematical physics. Its use here denotes a proposed family of indexed metrics and tolerances. Any empirical or physical interpretation requires an operational definition and evidence.

4.5 Subsystem D — Biological profile and participant safeguards

Formal component	Layer	Required tolerance / acceptance criterion	Functional purpose
Metabolic Profile Synchronizer	M3 observation interface	Instrument-specific accuracy, sampling rate, clock synchronisation, calibration records	Aligns physiological telemetry with experiment timestamps
Q(t) Data Model	M3 representation	Explicit variables, units, uncertainty, provenance	Represents selected biological measurements
Participant Isolation and Safety Monitor	Physical environment	Applicable safety standards and independent review	Monitors ordinary environmental and physiological risks
Consent and Data Governance Module	M0–M3	Explicit consent, access control, retention policy, revocation handling	Protects privacy and participant autonomy
Subjective-Experience Boundary Register	M0–M7+	No unsupported inference from telemetry to subjective experience	Prevents physiological data from being misrepresented as a complete model of consciousness

The Q-profile boundary

$$Q(t) = [q_1(t), q_2(t), \ldots, q_n(t)]^T$$

The components might represent heart rate, respiration, temperature, or other appropriately measured signals. Their meanings and units must be declared individually.

The model must not equate Q(t) with the entirety of subjective experience. Physiological telemetry is an observable data record; the subjective character of experience is a distinct philosophical and scientific problem.

The synchronizer preserves the integrity, timing, privacy, and provenance of measured profiles. It does not claim that consciousness can be extracted, transferred, or protected across hypothetical timelines.

4.6 Subsystem E — Physical computing and measurement infrastructure

Formal component	Layer	Required tolerance / acceptance criterion	Functional purpose
General-Purpose Computing Platform	M0–M6	Deterministic execution where required; tested resource limits	Executes the finite formal model
Reference Clock	M3	Traceable calibration and documented uncertainty	Provides timestamp reference
Environmental Sensors	M3	Device-specific calibration and measurement uncertainty	Records experimental conditions
Independent Logging System	M0–M3	Timestamped records and verified integrity	Supports reproducibility and post-run audit
Fault Containment Controller	Physical/software interface	Safe-state transition verified by testing	Stops unsafe or invalid operations
Simulation and Test Harness	M0–M7+	Reproducible inputs, documented assumptions, regression tests	Evaluates the model without claiming physical displacement

No numerical precision values are prescribed for generic equipment. The correct tolerances depend on the actual measurement task. Inventing a nanosecond tolerance, magnetic-field threshold, or dimensional specification without a defined physical mechanism would create the appearance of engineering certainty without a valid design basis.

SECTION 5 — OPERATIONAL RUNTIME CHECKLIST

5.1 Mission statement

The CRA runtime procedure initialises a formal model, verifies its consistency, computes a candidate fixed point, evaluates declared cross-layer mappings, and preserves an auditable record of the result.

This is not a validated firing sequence for physical chronological displacement. Terms such as ignition, engagement, and abort vector refer to software execution and experimental control unless a future physical mechanism is independently demonstrated.

STAGE I — PRE-IGNITION BOUNDARY ISOLATION

Objective: establish a well-defined state space, operator, constraint policy, and safe execution boundary.

☐ I-01 — Declare the domain. Define U, the set of admissible formal elements, and document the representation used to encode its members.

☐ I-02 — Construct the lattice. Verify that the selected state space supports the required order and join operations. For a powerset model, set L = P(U).

☐ I-03 — Initialise the zero-point state. Set $X_0 = \varnothing$.

☐ I-04 — Register the operator. Specify $\Gamma_C: L \to L$ as a total, well-defined mapping on the declared domain.

☐ I-05 — Verify monotonicity. Establish that $X \subseteq Y$ implies $\Gamma_C(X) \subseteq \Gamma_C(Y)$.

☐ I-06 — Test extensivity if required. Verify $X \subseteq \Gamma_C(X)$ for all states in scope. Do not infer extensivity from monotonicity alone.

☐ I-07 — Test idempotence if required. Verify $\Gamma_C(\Gamma_C(X)) = \Gamma_C(X)$ for all states in scope. If intended as a closure operator, all three closure properties must hold.

☐ I-08 — Verify mapping domains. Ensure every r_m,n, Act_n, and q_m,n has compatible domain and codomain declarations.

☐ I-09 — Enable logging and rollback. Preserve the initial state, operator version, constraint policy, software version, and all subsequent state changes.

☐ I-10 — Confirm safe-state behaviour. Demonstrate that invalid input, failed invariant, or lost software control stops computation without modifying external equipment.

Stage I acceptance criterion

$$\text{READY} = D \wedge M \wedge V \wedge S$$

D means the domain is declared, M means monotonicity is established, V means required invariants are verified, and S means the safe-state mechanism is available. This is a logical conjunction of readiness conditions, not a physical energy or activation threshold.

STAGE II — TARGET MATRIX TRACKING

Objective: compare candidate states and verify that the target satisfies the declared consistency policy.

☐ II-01 — Load the target representation. Identify candidate state s' and reference state s.

☐ II-02 — Validate feature maps. Confirm that $f_i(s)$ and $f_i(s')$ belong to the metric's declared domain.

☐ II-03 — Validate the metric. Confirm that d_i satisfies the mathematical properties required by its designation.

☐ II-04 — Declare the tolerance. Set $\varepsilon_i \geq 0$ based on intended use and documented uncertainty, not arbitrary tuning to force acceptance.

☐ II-05 — Evaluate the metric condition: $d_i(f_i(s), f_i(s')) \leq \varepsilon_i$.

☐ II-06 — Evaluate the equivalence condition. Test $s \sim_C s'$ independently. Do not infer transitivity from the threshold metric.

☐ II-07 — Check invariants. Confirm that the candidate satisfies all declared admissibility and consistency constraints.

☐ II-08 — Record the result. Store input states, metric value, tolerance, constraint-policy version, and acceptance or rejection outcome.

☐ II-09 — Reject ambiguous candidates. If a metric, domain, or equivalence test is undefined, return an explicit error rather than treating the state as accepted.

Stage II acceptance criterion

$$\text{TARGET_ACCEPTED iff } (s \sim_C s') \wedge [d_i(f_i(s), f_i(s')) \leq \varepsilon_i] \wedge C(s,s')$$

C(s,s′) denotes the explicitly defined consistency test. Passing this criterion establishes model-level acceptance only. It does not establish that a physical target timeline exists or can be reached.

STAGE III — INDEXICAL TRANSITION ENGAGEMENT

Objective: execute the formal transition, monitor convergence, and validate the resulting representation.

☐ III-01 — Confirm Stage I and Stage II acceptance. No transition is evaluated with invalid inputs.

☐ III-02 — Evaluate the next operator state: $X_{\alpha+1} = \Gamma_C(X_\alpha)$.

☐ III-03 — Handle limit stages mathematically: $X_\lambda = \bigcup_{\beta < \lambda} X_\beta$ for limit ordinals. A finite program must implement a bounded representation, not pretend to execute arbitrary transfinite ordinals physically.

☐ III-04 — Check convergence: $X_{\alpha+1} = X_\alpha$. If equality holds, record the fixed point. Otherwise, continue only within the declared computational bound.

☐ III-05 — Validate the result. Recheck all invariants and confirm that the output belongs to the declared state space.

☐ III-06 — Verify cross-level coherence: $Act_n(r_{m,n}(x)) = q_{m,n}(Act_m(x))$ for all required test inputs.

☐ III-07 — Run the metric filter. Confirm that the output satisfies defined distance and consistency conditions.

☐ III-08 — Preserve the original state. Store the pre-transition state and enough information to reproduce or reverse the software-level operation.

☐ III-09 — Publish the outcome accurately. Classify it as a computed fixed point, rejected state, iteration-limit event, or failed validation. Never label a software result as demonstrated temporal displacement.

Stage III acceptance criterion

$$\text{VALID_RESULT iff FixedPoint} \wedge \text{InvariantPass} \wedge \text{CoherencePass}$$

A valid result means that the formal model passed its declared checks. Physical chronology alteration is not an output of this criterion.

5.2 Emergency Abort Vector — Zero-Point Recovery

Trigger conditions:

• A required invariant fails.

• The operator returns an invalid state.

• Metric evaluation becomes undefined.

• Cross-layer coherence fails.

• The iteration bound is exceeded without convergence.

• The software loses integrity or the safe-state controller reports a fault.

Required response:

☐ Stop the current state-transition loop.

☐ Disable any dependent experimental control outputs.

☐ Mark the current run as aborted.

☐ Preserve the last valid state and fault record.

☐ Restore the model's initial state for a fresh run.

☐ Reinitialise the convergence monitor and audit ledger.

☐ Require a new validation pass before resuming.

$$X_{reset} := X_0 = \varnothing$$

This assignment resets the represented computational state. It does not erase the history of the run, reverse physical time, undo real-world events, or restore a person to an earlier timeline. The audit log must remain intact.

Abort-state pseudocode

```
def abort_to_zero_point(
    disable_outputs,
    preserve_audit_log,
    initialise_run_state
):
    """Software reset, not physical time reversal."""
    disable_outputs()
    preserve_audit_log()

    X0 = frozenset()
    initialise_run_state(X0)

    return X0
```

The implementation must make disable_outputs() and preserve_audit_log() safe, independently tested operations. A reset must not depend on the continued validity of the subsystem that triggered the abort.

5.3 Final engineering disposition

The CRA v4.0 model has a rigorous mathematical core when its domain, operator, order relation, and constraints are specified correctly. The complete-lattice framework supports a least-fixed-point theorem; transfinite iteration supplies a formal construction; metric filtering supplies a test for declared state similarity; and coherence equations constrain cross-layer representations.

These results do not establish a mechanism for time travel. The proposed higher strata, actuality maps, and Zodiacal Metric remain conceptual elements until given independently justified semantics and, where physical claims are intended, experimentally testable consequences.

Scientifically defensible development path

1. Formalise: specify U, L, Γ_C, and the complete constraint policy.

2. Prove: establish monotonicity and any additional closure or coherence properties claimed.

3. Implement: build a finite, reproducible software model and validate it against known test cases.

4. Connect to physics: derive a well-defined relationship to spacetime geometry and established physical laws rather than assuming one.

5. Test: identify a measurable prediction that distinguishes SSR from existing models.

6. Validate independently: require reproducible evidence before promoting any proposed physical mechanism from speculation to engineering.

FINAL BOUNDARY STATEMENT
The Least-Fixed-Point Chronological Realignment Array is presently a conceptual mathematical and computational architecture. Its formal operations can be implemented and studied, but a real, working machine that transfers an observer between timelines has not been established by SSR, order theory, or currently validated physics.

END OF DOCUMENT · SSR-CRA MODEL v4.0